

ABOUT

중학생 시절

미니게임 출시

직접 만든 게임을
실제로 배포

고등학교

디미고 (디지털미디어고)

게임 개발
독립 출시

대학교

한양대 바이오공학부

시스템 분석·모델링
사고방식 형성

현재

SSAFY 14기

LostArkSim
개발·운영 중

중학생 때 처음으로 게임을 직접 만들어 배포했고, 디미고에서 게임 개발을 이어갔습니다. 한양대 바이오공학부에서는 **시스템을 분해하고 모델링하는 사고 방식**을 다졌고, 이것이 지금의 시뮬레이션 개발 방향으로 자연스럽게 이어졌습니다. 연습용 프로젝트보다 **실제로 쓰이는 것**을 만드는 것을 원칙으로 삼고, 기술적 의사결정의 근거를 직접 검증하며 개발합니다.

프로젝트

LostArkSim

로스트아크 DPS / 스킬 로테이션 시뮬레이터

lostarksim.com
github.com/acapzed
acapz.dev (개발기)

"카더라 말고 데이터로" — 로스트아크 캐릭터의 스킬 로테이션을 Monte Carlo 방식으로 수천 회 시뮬레이션하여 DPS 분포를 산출하는 웹 애플리케이션. 현재 아카나 작업을 시작으로 개발 중이며, 연산 전체를 클라이언트 사이드에서 처리하도록 설계하여 서버 연산 비용을 실질적으로 0으로 만들었습니다. 디스코드 커뮤니티도 준비 중입니다.

서버 연산 → WASM	연산집약적 작업을 서버에서 처리하려면 고사양 서버가 필요했고, 비용과 대기시간 문제 가 심각했습니다. 클라이언트 사이드 WASM으로 전환하여 서버 연산 비용을 실질적으로 0으로 만들었습니다.
Go → Rust	WASM Worker 환경에서 하나의 캐릭터 세팅 정보(스탯·스킬 구성, 이하 profile)를 메모리 복사로 모든 Worker에 동일하게 공유 해야 했습니다. Go는 직접 메모리 접근이 불가능해 구조적 결함이 있었고, 직접 메모리 접근이 가능한 Rust로 전환했습니다.
MongoDB → PostgreSQL	추후 통계 처리에 유리한 구조 로 미리 전환했습니다. 관계형 DB의 집계·분석 쿼리 표현력을 선제적으로 확보하기 위한 결정이었습니다.
CRA → Vite	빌드 속도가 너무 느려 전환했습니다. 이후 CRA가 공식 deprecated 되어 결과적으로도 올바른 판단이었습니다.



시뮬레이션 결과 화면 — DPS 분포, 스킬별 데미지 비중, 스킬 시전 순서

Rust

WebAssembly

TypeScript

PostgreSQL

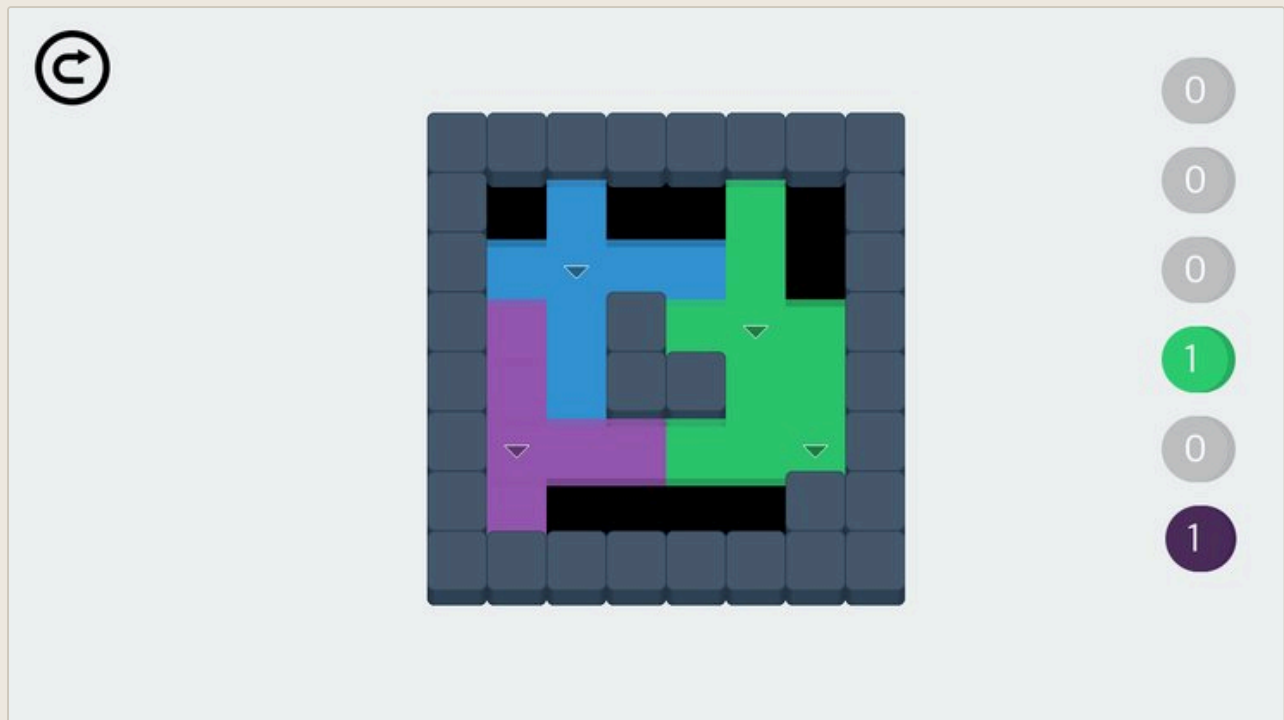
Docker

Nginx

GitHub Actions

Cloudflare

DigitalOcean



상하좌우 2칸까지 퍼져나가는 향수를 적재적소에 배치해 모든 빈칸을 채우는 퍼즐 게임입니다. 같은 색상의 향수끼리는 섞일 수 있지만, 다른 색상끼리는 섞이면 안 된다는 규칙 위에서 48개 난이도를 직접 기획하고 구현하여 Google Play에 출시했습니다. 기획을 여러 차례 갈아엎으면서도 완성까지 밀어붙인 첫 번째 완주 경험이었습니니다.

기술 블로그 — ACAPZ.DEV

스택 전환의 이유

CRA→Vite, Go→Rust WASM, MongoDB→PostgreSQL 각 전환의 배경과 의사결정 과정 기록.

왜 연습용 CRUD가 아닌가

실서비스 프로젝트를 통해서만 부딪히는 문제들과 그것이 성장에 어떻게 작용하는 지에 대한 개발 철학.

기술 스택

● Rust

● WebAssembly

● TypeScript

● Go

● Python

● Java / Kotlin

● PostgreSQL

● Redis

● Docker

● Spring Boot

● Cloudflare

● GitHub Actions